Beyond Lines: A Factorized Deep Hough Transform for Parametric Curve Detection

Joy Bose*
Independent Researcher, Bengaluru, India
joy.bose@ieee.org

Abstract

We extend the Deep Hough Transform (DHT), which detects semantic lines by aggregating learned features along candidate lines into a two-dimensional parameter space, to higher-dimensional parametric curve families. The extension is hard because the accumulator for a family with d parameters has $O(B^d)$ cells for B bins per axis. We propose a factorized deep Hough transform, a differentiable feature-space voting cascade that splits the parameter vector into a spatial anchor and a shape. A coarse two-dimensional accumulator votes for anchors, and a dense shape accumulator is built only around the top-k anchors, so memory falls from $O(B^d)$ to $O(B^{d_1} + kB^{d_2})$. Lines are the degenerate case with no second stage. We also define a curve agreement score that extends the EA line similarity to parametric curves, and a synthetic benchmark of parabolas, circles and ellipses with noise, occlusion and distractors.

We evaluate with thresholds selected on a validation split and the test split scored once, and we give the geometric baselines the same treatment. Under this protocol the choice of Stage-1 pooling over probe shapes matters a great deal: max pooling raises parabola F from 0.51 to 0.70 over log-sum-exp (3 seeds). With it, the cascade ties a tuned RANSAC fit on parabolas (0.706 vs. 0.702) and circles (0.743 vs. 0.742) and is slightly behind it on ellipses (0.382 vs. 0.410), while direct regression and query-based heads reach 0.15 and 0.13 on parabolas. So the cascade is competitive with classical fitting rather than better than it, and its practical advantage is memory: at $B = 32$ a dense $d = 5$ accumulator would need about 4 GB (computed, not instantiated) against a measured 236 KB for the cascade. We locate the remaining error in Stage 1. Running Stage 2 at the ground-truth anchors gives F = 0.938 against 0.706, and curves whose anchors lie within about 8 pixels of each other lose about 0.2 F because their Stage-1 peaks merge. We also report a TuSimple lane-detection stress test, which we treat as a feasibility check rather than a competitive result. Code and the benchmark generator are released.

1 Introduction

The Hough transform [1, 2] detects a parametric shape by letting evidence vote for every shape that could explain it. For lines, each edge pixel votes for all lines through it, and peaks in the (θ, r) space are detections. The Deep Hough Transform (DHT) [10] moves this into a network: instead of voting edge pixels it averages learned CNN features along every candidate line, finds peaks in the parameter space with a small convolutional head, and trains everything end to end.

Classical Hough theory has been extended to circles, ellipses and algebraic curves [3, 8, 9], but the feature-space formulation has stayed with lines [10, 11, 12]. The reason is cost. A line has two

*Code, benchmark generator and configurations: https://github.com/joyboseroy/curve-hough.

parameters, so its accumulator is a small map. A vertex-form parabola or a circle has three, an ellipse five, and a dense accumulator with B bins per axis has B^d cells, each holding a C-channel feature. Deep curve detectors therefore moved to direct parameter regression [14, 15, 16], which gives up the explicit evidence surface that voting provides.

Our approach follows the multi-stage ellipse detectors of the classical literature [4]: find where the curve is first, then what shape it has. We write the parameters as an anchor and a shape. Stage 1 votes over anchors only, pooling over a small set of probe shapes. Stage 2 votes over shape only, and only around the k strongest anchors. The cost of the accumulators drops from $O(B^d)$ to $O(B^{d_1} + kB^{d_2})$.

The detector is family-conditional: the target family and its parameterization are specified in advance, as in classical Hough detection. The question we ask is whether DHT-style feature-space voting can be made tractable once a family of dimension $d > 2$ is given. It is not an attempt at open-world shape discovery.

Parameter-space factorization itself is not new, since multi-stage Hough methods have a long history. What we contribute is its formulation as a differentiable deep-feature voting cascade, in which learned image features are aggregated over conditional curve hypotheses and trained through peak losses in every accumulator. We use validation-selected thresholds throughout and report an ablation of the Stage-1 pooling operator, which turns out to matter a great deal.

The contributions are as follows.

- **Method.** A factorized deep Hough transform for curve families with a spatial anchor, with lines as the degenerate case (Section 3), and a curve-EA similarity that extends the EA line score (Section 4).
- **Protocol.** A benchmark generator and an evaluation protocol with validation-selected thresholds, matched baselines that include tuned RANSAC for parabolas, circles and ellipses, and ablations (Sections 5 and 6).
- **Diagnosis.** A decomposition of the cascade's error into Stage 1 and Stage 2 using an oracle-anchor ceiling, top-k anchor recall and an anchor-separation sweep (Section 6.3). Stage 1 is the bottleneck, and nearby anchors are a quantified failure region.
- **Scope of the claim.** At $d = 3$ the cascade matches a tuned RANSAC fit and at $d = 5$ it is somewhat behind. What it adds is differentiable feature-space voting with memory that grows with the shape dimension only (Section 6.5). We do not claim better accuracy than classical fitting.
- **Real-image stress test.** A TuSimple evaluation with a structural split into near-straight and curved lanes (Section 7), reported with its limitations.

2 Related Work

Classical Hough transforms for curves. The Hough transform was introduced for bubble chamber photographs [1] and formalized for lines with the angle-radius parameterization [2]. Ballard [3] generalized voting to arbitrary shapes. Dimensionality is the main obstacle. Randomized [5] and probabilistic [6] variants subsample votes, adaptive variants refine the grid [7], and multi-stage methods split the parameter space, for example detecting ellipse centers before axes [4]. Hough methods for special algebraic curves have been used for anatomical profile extraction in CT [8, 9]. Our cascade is a differentiable, feature-space version of this decomposition. For ellipses, RANSAC with a direct conic fit [18, 19] is a strong classical alternative, and we compare against it.

Deep Hough methods. DHT [10] aggregates CNN features along candidate lines into (θ, r) space, detects peaks with convolutions, and introduces the EA metric and the NKL dataset. Lin et al. [11] add trainable Hough line priors inside a CNN. HoughLaneNet [12] applies a hierarchical DHT to lane detection, but the voting is line-based and curves are recovered afterwards by dynamic convolution. Deep Hough voting is also used for 3D object centroids [13]. We are not aware of prior feature-space voting over curve parameter spaces.

Deep curve detection by regression. PolyLaneNet [14] regresses polynomial coefficients, LSTR [15] predicts polynomial lane shapes with a transformer, BezierLaneNet [16] predicts cubic Bezier control points, and BSNet [17] uses B-splines. These avoid the accumulator but also give up an explicit evidence surface. Our regression and DETR-style baselines [21] stand in for this family at small scale.

3 Method

3.1 Deep Hough voting

Let $\mathbf{X} \in \mathbb{R}^{C \times H \times W}$ be encoder features. For a curve family with parameters $\mathbf{p} \in \mathcal{P} \subset \mathbb{R}^d$, let $\Omega(\mathbf{p})$ be the pixel set of the curve $C(\mathbf{p})$ clipped to the image. Voting computes

$$\mathbf{Y}(\mathbf{p}) = \frac{1}{|\Omega(\mathbf{p})|} \sum_{i \in \Omega(\mathbf{p})} \mathbf{X}(i), \tag{1}$$

which for lines and a quantized $\mathcal{P}$ is the DHT of Zhao et al. [10] up to normalization. A small convolutional head on $\mathbf{Y}$ gives detections. With B bins per axis, $\mathbf{Y}$ has $C \cdot B^d$ entries.

3.2 Parameter factorization and scope

We write $\mathbf{p} = (\mathbf{a}, \mathbf{s})$ with an *anchor* $\mathbf{a} \in \mathbb{R}^{d_1}$ (a spatial point: parabola vertex, circle or ellipse center) and a *shape* $\mathbf{s} \in \mathbb{R}^{d_2}$ (curvature, radius, axes and orientation). For a vertex-form parabola $y = s(x - a_1)^2 + a_2$ we have $\mathbf{a} = (a_1, a_2)$ and $\mathbf{s} = s$. For a circle, $\mathbf{a}$ is the center and $\mathbf{s} = r$. For an ellipse, $\mathbf{a}$ is the center and $\mathbf{s} = (r_x, r_y, \phi)$.

The method assumes that the curve family is known and that it has a spatially meaningful anchor. Both assumptions are shared with classical Hough voting, which also searches a fixed hypothesis space, and both are by design: the accumulator needs a parameterization to index. A family without such an anchor would need a different factorization, and we do not address it.

Lines as the degenerate case. A line has no anchor separate from its own parameters, so $d_1 = d = 2$, $d_2 = 0$, and Stage 2 is empty. The cascade is then single-stage voting over (θ, ρ), which is the original DHT. Lines use the same encoder, training loop and detection code as the other families, and differ only in whether Stage 2 is built. This lets the real-image experiment in Section 7 use one cascade for both near-straight and curved lanes.

3.3 Stage 1: marginal anchor voting

Stage 1 builds an accumulator over anchors only, pooling over a small probe set $\tilde{\mathcal{S}} = \{\mathbf{s}_1, \ldots, \mathbf{s}_m\}$ of shapes:

$$\mathbf{Y}_1(\mathbf{a}) = \operatorname*{pool}_{\mathbf{s} \in \tilde{\mathcal{S}}} \frac{1}{|\Omega(\mathbf{a}, \mathbf{s})|} \sum_{i \in \Omega(\mathbf{a}, \mathbf{s})} \mathbf{X}(i). \tag{2}$$

Algorithm 1 Factorized deep Hough cascade (inference)

$\mathbf{X} \leftarrow$ Encoder(image)
for each anchor bin $\mathbf{a}$ **do**
  $\mathbf{Y}_1(\mathbf{a}) \leftarrow \max_{\mathbf{s}\in\tilde{\mathcal{S}}}\ \mathrm{mean}_{i\in\Omega(\mathbf{a},\mathbf{s})}\mathbf{X}(i)$ ▷ Eq. 2
end for
$\mathbf{P}_1 \leftarrow \mathrm{ConvHead}_1(\mathbf{Y}_1)$; $\{\hat{\mathbf{a}}_1,\dots,\hat{\mathbf{a}}_k\} \leftarrow$ TopK-NMS($\mathbf{P}_1$)
for each retained anchor $\hat{\mathbf{a}}_j$ **do**
  for each shape bin $\mathbf{s} \in \mathcal{S}_{\mathrm{dense}}$ **do**
    $\mathbf{Y}_2^{(j)}(\mathbf{s}) \leftarrow \mathrm{mean}_{i\in\Omega(\hat{\mathbf{a}}_j,\mathbf{s})}\mathbf{X}(i)$ ▷ Eq. 3
  end for
  $\mathbf{P}_2^{(j)} \leftarrow \mathrm{ConvHead}_2(\mathbf{Y}_2^{(j)})$; $\hat{\mathbf{s}}_j \leftarrow$ SoftArgmaxPeak($\mathbf{P}_2^{(j)}$)
end for
return $\{(\hat{\mathbf{a}}_j, \hat{\mathbf{s}}_j, \mathrm{score}_j)\}_{j=1}^{k}$

$\mathbf{Y}_1$ has $C \cdot B_a^2$ entries. A small 2D convolutional head turns it into an anchor map $\mathbf{P}_1$. The pooling operator is a design choice that matters. We compare log-sumexp and max in Section 6.2; mean pooling was not evaluated. Max pooling reads as "some probe shape fits here", which is the right question for localizing an anchor when the shape is unknown, and it is our default. The probe set has $m = 6$ shapes for parabolas and circles, and $m = 27$ for ellipses (three values per shape axis).

3.4 Stage 2: conditional shape voting

For each of the top-k anchor peaks $\hat{\mathbf{a}}_j$ (3×3 local maxima of $\mathbf{P}_1$, ranked by score), Stage 2 builds a dense accumulator over shape only:

$$\mathbf{Y}_2^{(j)}(\mathbf{s}) = \frac{1}{|\Omega(\hat{\mathbf{a}}_j, \mathbf{s})|} \sum_{i\in\Omega(\hat{\mathbf{a}}_j,\mathbf{s})} \mathbf{X}(i), \qquad \mathbf{s} \in \mathcal{S}_{\mathrm{dense}}. \tag{3}$$

A convolutional head and a peak selection give the full detection $(\hat{\mathbf{a}}_j, \hat{\mathbf{s}}_j)$ with a score. Soft-argmax over the local neighborhood gives sub-bin precision in both stages. Re-voting on a finer grid around each detection, in the manner of coarse-to-fine classical Hough [7], is not implemented, and nothing reported here uses it.

Proposition 1 (Accumulator size)**.** *With $\mathbf{a} \in \mathbb{R}^{d_1}$, $\mathbf{s} \in \mathbb{R}^{d_2}$, $d_1 + d_2 = d$ and B bins per axis, the dense accumulator of Eq. 1 has $O(B^d)$ entries, and the cascade has $O(B^{d_1} + kB^{d_2})$ entries, where k is the number of retained anchors.*

Proof. Stage 1 accumulates over $\mathbf{a}$ only. The probe set has size m independent of B, so Stage 1 holds B^{d_1} cells per channel. Stage 2 is built at k anchors and each holds B^{d_2} cells per channel. ■

The bound is about accumulator memory. It says nothing about accuracy, and the factorization is lossy: Stage 2 only sees anchors that Stage 1 retained. Section 6.3 measures how much that costs.

3.5 Training

As in Zhao et al. [10], ground-truth curves are mapped into each accumulator, smoothed with a Gaussian, and each stage is trained with binary cross-entropy. Stage 2 targets are conditioned on the ground-truth anchors (teacher forcing) and on detected anchors within a matching radius. The voting layers are parameter-free gather-and-mean operations, so the encoder and both heads train

end to end. The two losses are summed with equal weight. Stage-1 targets use a Gaussian of $\sigma = 1.5$ bins and a positive-class weight of 8, and Stage-2 targets use $\sigma = 1.0$ bins. All runs use Adam with learning rate 2×10^{-4}, batch size 8 and 10 epochs. The anchor grid has $B_a = 32$ bins per axis and the dense shape grid $B_s = 48$ bins, with $C = 32$ channels and $k = 4$.

4 The Curve-EA Score

The EA score of Zhao et al. [10] multiplies an angular and a midpoint distance term, both specific to lines. For curves c_1, c_2 we sample T points $\{u_t^{(1)}\}, \{u_t^{(2)}\}$ uniformly in arc length inside the image and define a normalized symmetric Chamfer term

$$\mathcal{S}_d = 1 - \min\left(1, \frac{\mathrm{CD}(\{u^{(1)}\}, \{u^{(2)}\})}{D_{\max}}\right), \tag{4}$$

with $D_{\max}$ the image diagonal times a constant, and a tangent agreement term

$$\mathcal{S}_\theta = 1 - \frac{1}{T} \sum_t \frac{\angle\big(\tau_1(u_t^{(1)}), \tau_2(\pi(u_t^{(1)}))\big)}{\pi/2}, \tag{5}$$

where τ are unit tangents and $\pi(\cdot)$ is nearest-point correspondence. The score is $\mathcal{S} = (\mathcal{S}_d \cdot \mathcal{S}_\theta)^2$ in $[0, 1]$. Predictions and ground truth are matched by the Hungarian algorithm and precision, recall and F are averaged over a grid of similarity thresholds, as in Zhao et al. [10]. Because the grid includes lenient thresholds, a detection whose anchor is a few pixels off can still count as a match. This matters when reading the anchor-recall numbers below.

5 Evaluation Protocol

Benchmark. Images are 64×64 in all reported results (the generator supports 128×128). Each image has one to three curves from one family, drawn with stroke width 2, plus Gaussian pixel noise ($\sigma_n = 0.08$ unless varied), up to two random occluding rectangles (occ_max = 2), and up to three distractor line segments. Figure 1 shows examples.

Splits and seeds. Each run trains on 1,000 images and uses a validation split of 200 images and a test split of 200 images, all drawn from the same generator with different seeds. The seed of a run changes model initialization, data order and the three image draws, so the standard deviations we report over seeds include data variance. Differences below about 0.02 F are within noise at this size: two runs of the same configuration and seed gave 0.706 and 0.690.

Threshold selection. A detection is kept if its score is at least a threshold τ. We sweep τ over $\{0.05, 0.10, \ldots, 0.95\}$ on the validation split, pick the best, and score the test split once at that value. As a check we also run a test-set sweep and report how much it would have overstated F. Across all runs reported here the gap is between 0.000 and 0.011, so the choice of protocol changes the numbers little.

Baselines. All baselines use the same images, metric and splits.

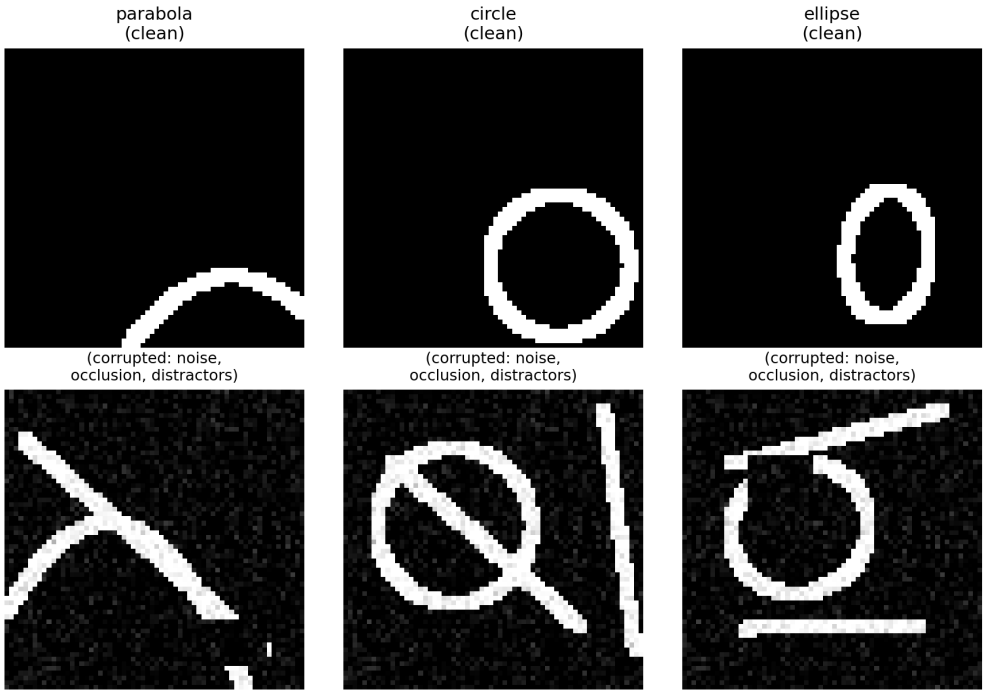


Figure 1: Benchmark images, clean (top) and corrupted (bottom: noise, occlusion, distractor segments), for each curve family.

- *RANSAC* on Sobel edge points: a 3-point minimal solver for parabolas and circles, and a 5-point conic fit with Sampson distance for ellipses [19, 20]. *Tuned* RANSAC keeps candidates whose inlier count passes a threshold chosen on validation, which is the treatment the learned detectors get. *Untuned* RANSAC always keeps the top four candidates.
- *Classical dense Hough* on Sobel edges, with a tuned accumulator threshold. It runs at 16 anchor bins and 16 shape bins, coarser than the cascade's 32, so it is handicapped. It is not run for ellipses, where a dense 5D accumulator is infeasible at useful resolution.
- *Direct regression*: the same encoder and an MLP head that predicts up to three parameter vectors, trained with a permutation-matched L1 loss.
- *DETR-lite queries*: the same encoder and a small DETR-style decoder with learned queries and Hungarian matching.

The two learned baselines get the same 1,000 training images and 10 epochs as the cascade. They are small models trained with a small budget, so their numbers describe this regime and should not be read as a bound on what regression or query heads can do with more data.

6 Results

6.1 Comparison with baselines

Table 1 gives F on the corrupted benchmark for seed 0. All methods see the same 200 test images.

Three observations follow. First, the cascade with max pooling ties a tuned RANSAC fit at $d = 3$ on both parabolas and circles. On ellipses it is 0.03 behind. We read the ellipse gap as real

Table 1: F on the corrupted benchmark, 64×64, seed 0, thresholds selected on validation. "n.r." means not run. Differences below about 0.02 are within test noise. Over three seeds the parabola cascade with max pooling scores 0.696 ± 0.011 (Table 2).

Method	**Parabola** ($d = 3$)	**Circle** ($d = 3$)	**Ellipse** ($d = 5$)
Factorized DHT, max pool (ours)	0.706	0.743	0.382
Factorized DHT, log-sumexp	0.519	n.r.	0.319
RANSAC, tuned	0.702	0.742	0.410
RANSAC, untuned (top 4)	0.551	0.584	0.383
Classical dense Hough, tuned	0.609	n.r.	infeasible
Direct regression	0.151	n.r.	n.r.
DETR-lite queries	0.131	n.r.	n.r.

Table 2: Stage-1 pooling over probe shapes, parabola, corrupted benchmark, mean $\pm$ std over seeds. Anchor recall at $k = 4$ uses a tolerance of one anchor bin (about 2 px).

Pooling	**val F**	**test F**	**anchor recall@4**
max	0.697 ± 0.024	0.696 ± 0.011	0.765
log-sumexp	0.533 ± 0.016	0.511 ± 0.006	0.494

but small, since it sits near the test-noise level and we have one seed. Second, tuning matters for the baselines as much as for the cascade. Untuned RANSAC loses about 0.15 F on parabolas and circles because it always emits four candidates. Third, the regression and query heads score 0.15 and 0.13 on parabolas, about a fifth of the geometric methods. We have not run them on circles and ellipses. We suspect the gap reflects the difficulty of predicting exact parameters from pooled features when several curves, occlusion and distractors share the image, but a small training budget also contributes, and this experiment does not separate the two.

6.2 Stage-1 pooling

Table 2 varies the pooling operator in Eq. 2 on parabolas over three seeds.

Max pooling is better by about 0.18 F and by 0.27 in anchor recall, and the effect is far larger than the seed variation. Max was chosen on validation F using seeds 1 and 2 (0.710 against 0.539) and then confirmed with seed 0. Log-sumexp softens the max over probe shapes, which appears to blur the anchor map when several probe shapes fit partially.

6.3 Where the cascade loses accuracy

Stage 2 only runs at anchors that Stage 1 retains, so a missed or misplaced anchor cannot be repaired. We measure this in three ways, all for parabolas with max pooling and seed 0.

Oracle-anchor ceiling. Running Stage 2 at the ground-truth anchors, with Stage 1 removed, gives F = 0.938, against 0.706 for the real pipeline. Stage 1 costs 0.232 F, and Stage 2 is not the limiting stage.

Top-k anchor recall. For each k we count ground-truth curves whose anchor is matched, one to one, by one of the k strongest Stage-1 peaks within a tolerance in anchor bins (Table 3, Figure 2). Exact-bin recall is low (0.29 at $k = 4$), but with a one-bin tolerance it is 0.80, and most of the gain

Table 3: Top-k Stage-1 anchor recall, parabola, max pooling, seed 0. Tolerance is in anchor bins (1 bin is about 2 px).

Tolerance	$k = 1$	$k = 2$	$k = 4$	$k = 8$
0 bins	0.189	0.263	0.288	0.291
1 bin	0.452	0.684	0.796	0.811
2 bins	0.472	0.724	0.883	0.903

Table 4: Two curves with anchors sep pixels apart, noise only, threshold 0.55, parabola, max pooling.

sep (px)	0	2	4	6	8	12	16	24
anchor recall@4	0.475	0.482	0.492	0.525	0.568	0.812	0.870	0.912
F	0.584	0.588	0.584	0.582	0.640	0.746	0.779	0.809

comes between $k = 1$ and $k = 4$. This is a strict localization measure and not a bound on recall in the F metric, which accepts detections whose anchor is a few pixels off.

Anchor separation. We generate images with two curves whose anchors are sep pixels apart and with different shape parameters, and measure anchor recall at $k = 4$ and F (Table 4). Recall and F are flat below about 8 px and rise above it. Anchors closer than about four anchor bins fall inside the 3×3 peak window and merge into one Stage-1 peak, which costs roughly 0.2 F. This is a concrete failure region of the current design, and the one to fix first.

6.4 Number of retained anchors

Table 5 varies the number k of Stage-1 peaks passed to Stage 2 for parabolas with max pooling and seed 0. One trained model is evaluated at all four values of k, so the rows differ only in how many anchors Stage 2 sees. Probe-set size, noise level, occlusion level and mean pooling were not evaluated (Section 9).

Going from $k = 1$ to $k = 2$ adds 0.12 F and going to $k = 4$ adds a further 0.006, which is within noise, and $k = 8$ adds nothing. Anchor recall keeps rising slightly (0.786 to 0.804) while F does not. With one anchor, recall is low because images hold up to three curves. Beyond two anchors the extra candidates are mostly duplicates or near-misses of curves already found, and they lower precision as much as they raise recall. This agrees with the diagnosis above: more retained peaks do not repair a mislocalized or merged anchor, so the fix has to be in Stage 1 itself and not in k.

Two runs of the same configuration and seed gave 0.706 (Table 1) and 0.690 here, a spread of 0.016 that we have not traced, probably nondeterministic GPU kernels. We therefore treat single-run differences below about 0.02 as noise throughout.

6.5 Accumulator memory

Table 6 gives accumulator sizes in float32 at the configuration above ($C = 32$, $B = 32$ bins per axis). The factorized sizes are measured. The dense $d = 5$ size is arithmetic, because the accumulator is not instantiated. No dense accumulator was trained for $d \geq 4$ in this work, so the comparison below is about memory and not about accuracy.

From $d = 3$ to $d = 5$ the dense accumulator grows by $32^2 = 1{,}024\times$ while the factorized one grows 1.6×, because the anchor stays two-dimensional and only the shape grows. A dense $d = 5$

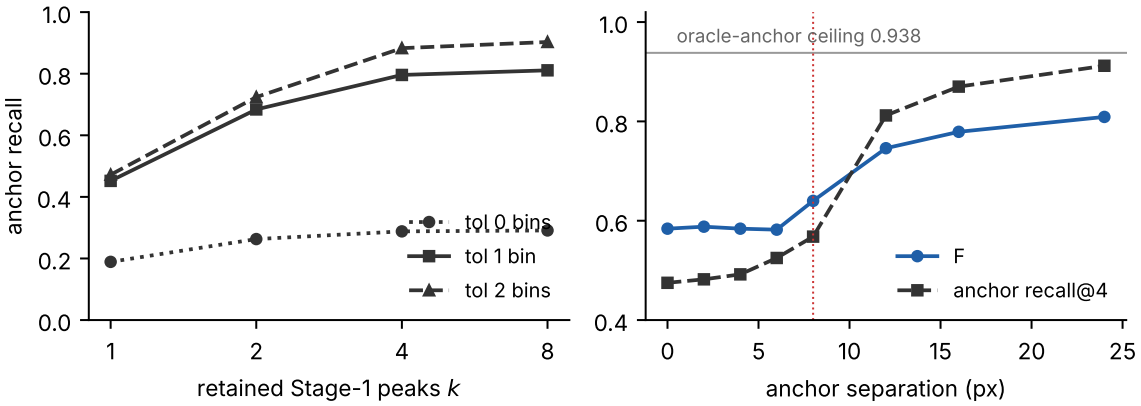


Figure 2: Stage-1 diagnostics for parabolas with max pooling (Tables 3 and 4). Left: top-k anchor recall for three tolerances. Right: F and anchor recall as two anchors approach each other. The dotted line marks 8 px.

Table 5: Retained anchors k, parabola, corrupted benchmark, max pooling, seed 0, one trained model. Thresholds are selected on validation. Anchor recall uses a tolerance of one bin. This model scored 0.690 at $k = 4$, while the run in Table 1 with the same seed scored 0.706 (see text).

k	val F	test F	P	R	anchor recall	threshold
1	0.522	0.560	0.828	0.423	0.446	0.05
2	0.649	0.684	0.751	0.628	0.666	0.55
4	0.674	0.690	0.666	0.717	0.786	0.55
8	0.671	0.684	0.652	0.719	0.804	0.55

accumulator of about 4 GB would not fit alongside a network on a typical single GPU, which is the practical reason dense voting stops at about $d = 3$. Classical Hough avoids it by coarse grids or by multi-stage schemes, which is the same idea in non-differentiable form. The measured ellipse results show that the cascade still works at $d = 5$, with F 0.382 against 0.410 for tuned RANSAC. They do not show that it beats a dense accumulator in accuracy, since we did not run one.

Remark. Eq. 1 depends only on the multiset of features over $\Omega(\mathbf{p})$, so the voting layer needs no ordering or positional encoding. It is a fixed gather-and-mean.

7 TuSimple Stress Test

We also ran the cascade on TuSimple [22], a dashcam lane dataset, letterboxed to 128×128. This is a feasibility check on real photographs. It is not a competitive lane-detection benchmark, and the results below are weaker in evidence than the synthetic ones above: they come from one run per setting, they predate the validation-threshold protocol of Section 5, and we have not evaluated the published lane detectors or the DHT-based lane method of related work under our metric. Those methods report the official TuSimple accuracy, which our curve-EA F is not comparable to, so we give no side-by-side numbers.

Table 6: Accumulator memory (float32), dense versus factorized, $C = 32$, $B = 32$. Dense sizes for $d = 5$ are computed, not instantiated.

	Dense $O(B^d)$	**Factorized**	**Ratio**
$d = 3$ (parabola), Stage 1 128 KB + Stage 2 24 KB	4.00 MB	152 KB	26.9×
$d = 5$ (ellipse)	4.00 GB	236 KB	17,772×

Table 7: TuSimple, held-out data, one run per setting. Near-straight lanes use the line family and curved lanes the parabola family. The RANSAC baseline uses the same Sobel edges and the same split but, like the untuned RANSAC in Table 1, keeps a fixed number of candidates.

Regime	**% of lanes**	**F (ours)**	**F (RANSAC, untuned)**
Near-straight (line)	64.9%	0.50	0.25
Curved (parabola)	35.1%	0.09 to 0.18	not run

7.1 A structural split

TuSimple labels lanes as polylines. We fit each lane to a parabola $x = a(y - y_0)^2 + x_0$ by least squares. Of 10,889 lanes with at least three labeled points, 3,823 (35.1%) give a numerically stable fit and 7,066 (64.9%) are close to straight, so the vertex-form conversion is unstable. We checked this by bounding the fitted vertex against the image extent, not by a curvature threshold alone, which let unstable fits through. We send near-straight lanes to the line family (no Stage 2) and curved lanes to the parabola family, and train and evaluate the two as separate models. A single detector that handles both regimes in one image is not implemented, and the two-model setup is a limitation.

A second issue appeared when we inspected Stage-1 outputs on real data. A parabola fit can be stable while its vertex lies outside the photographed scene. Our anchor grid covers only the image, so such targets cannot be learned. We drop lanes whose transformed vertex is more than half an image dimension outside the frame. On held-out curved lanes this roughly doubled F relative to training on the unfiltered set, and training on the smaller filtered set did better than training on the larger one.

7.2 Results

On near-straight lanes the line model reaches F = 0.50 (mean matched similarity 0.58, median 0.67), against 0.253 for the RANSAC line baseline. The synthetic results show that an untuned RANSAC loses about 0.15 F relative to a tuned one (0.551 against 0.702), so the factor of two over RANSAC here is probably an overstatement of the real advantage. A tuned RANSAC line baseline on TuSimple is a necessary rerun.

On curved lanes the result is weak and varies from run to run (F between 0.09 and 0.18). The Stage-1 heatmaps on real images are sharp and confident, unlike the diffuse synthetic ellipse case, so the cause does not look like a capacity limit of the voting. We suspect the small training pool (about 1,500 to 1,800 images after filtering) and the off-frame vertices the filter does not remove, but we have not tested either. We did not average over seeds.

Table 8: Hough voting and transformer cross-attention as read-outs that differ in where the hypothesis space and attention weights come from.

	Deep Hough voting (this work)	**Query decoding (DETR-lite)**
Hypothesis space	Fixed geometric parameterization	Learned embeddings
Attention weights	Set by geometry (in support or not)	Learned softmax similarity
Support	Sparse, exactly the curve's pixels	Dense, all tokens
Accumulator	Indexed by $(\mathbf{a}, \mathbf{s})$	Latent

8 Discussion: Voting as Geometry-Constrained Attention

Eq. 1 is a cross-attention read-out in which hypothesis $\mathbf{p}$ attends to pixel tokens with a fixed attention map: weight $1/|\Omega(\mathbf{p})|$ on the curve's pixels and zero elsewhere. A DETR decoder learns this map. That gives it flexibility and removes the geometric prior. Table 8 summarizes the contrast. Our parabola numbers (Table 1) are consistent with the prior helping at small scale, but, as noted above, they do not isolate the prior from the training budget.

This suggests a spectrum from fixed geometric attention, through geometric supports with learned per-pixel weights, to fully learned hypothesis embeddings. The last would let voting extend to hypothesis spaces that are not geometric, which we leave open.

9 Limitations

- **Family-conditional detection.** The target family is specified at inference time. This is deliberate and matches classical Hough voting, which also needs a parameterization of what it looks for, but it excludes open-world shape discovery and free-form curves. The factorization also needs a spatially meaningful anchor. A family-agnostic system could treat the family as another discrete hypothesis variable or learn a library of hypothesis spaces, which we do not attempt.

- **Stage 1 is the bottleneck.** The oracle ceiling of 0.938 against 0.706 shows that most of the remaining error is anchor localization. Anchors within about 8 px merge and cost about 0.2 F. Coarse-to-fine refinement and a multi-peak Stage 2 are the planned fixes and neither is implemented.

- **No accuracy advantage over classical fitting.** The cascade ties tuned RANSAC at $d = 3$ and is slightly behind at $d = 5$. Its benefit is differentiable feature-space voting and low accumulator memory. We did not train or evaluate a dense accumulator at $d \geq 4$, so the memory ratio is not an accuracy comparison.

- **Seeds and coverage.** The baseline comparison in Table 1 is one seed. Only the pooling ablation has three seeds. Circle and ellipse have no ablations, and the learned baselines were run for parabolas only. The ablations cover pooling and the number of retained anchors on parabolas only. Probe-set size, noise level, occlusion level and mean pooling were not evaluated.

- **Scale and data.** All main results are at 64×64 with 1,000 training images and 10 epochs. The benchmark is synthetic, and trends at 128×128 are untested. The regression and query baselines are small and briefly trained, so their gap to the geometric methods is specific to this budget.

- **Real images.** The TuSimple results are single runs predate the validation-threshold protocol, use an untuned RANSAC baseline, and handle near-straight and curved lanes with two separate models. Published lane detectors are not evaluated.

10 Conclusion

A deep Hough transform need not enumerate the full curve parameter space. If a curve family admits a spatial anchor, feature-space voting can be factorized into anchor localization followed by conditional shape voting, which reduces accumulator memory from $O(B^d)$ to $O(B^{d_1} + kB^{d_2})$ and makes DHT-style voting memory-feasible for families beyond lines. With max pooling over probe shapes, the cascade matches a tuned RANSAC fit on synthetic parabolas and circles, is slightly behind on ellipses, and needs 236 KB of accumulator memory at $d = 5$ where a dense accumulator would need about 4 GB. Its remaining error sits in Stage 1: Stage 2 reaches F = 0.938 at the ground-truth anchors, and nearby anchors that merge into one peak are the concrete failure mode. Detection of an unspecified family is outside the scope of this work. Natural next steps are an improved Stage 1, robustness sweeps over noise and occlusion, and a comparison with published lane detectors under their own metric. Code, configurations and the benchmark generator are released with the commands for every table.

References

[1] P. V. C. Hough. Method and means for recognizing complex patterns. US Patent 3,069,654, 1962.

[2] R. O. Duda and P. E. Hart. Use of the Hough transformation to detect lines and curves in pictures. *Commun. ACM*, 15(1):11–15, 1972.

[3] D. H. Ballard. Generalizing the Hough transform to detect arbitrary shapes. *Pattern Recognition*, 13(2):111–122, 1981.

[4] H. K. Yuen, J. Illingworth, and J. Kittler. Detecting partially occluded ellipses using the Hough transform. *Image Vis. Comput.*, 7(1):31–37, 1989.

[5] L. Xu, E. Oja, and P. Kultanen. A new curve detection method: randomized Hough transform. *Pattern Recognit. Lett.*, 11(5):331–338, 1990.

[6] N. Kiryati, Y. Eldar, and A. M. Bruckstein. A probabilistic Hough transform. *Pattern Recognition*, 24(4):303–316, 1991.

[7] J. Illingworth and J. Kittler. The adaptive Hough transform. *IEEE TPAMI*, 9(5):690–698, 1987.

[8] M.-L. Torrente and M. C. Beltrametti. Almost-vanishing polynomials and an application to the Hough transform. *J. Algebra Appl.*, 13(8):1450057, 2014.

[9] A. M. Massone, A. Perasso, C. Campi, and M. C. Beltrametti. Profile detection in medical and astronomical images by means of the Hough transform of special classes of curves. *J. Math. Imaging Vis.*, 51(2):296–310, 2015.

[10] K. Zhao, Q. Han, C.-B. Zhang, J. Xu, and M.-M. Cheng. Deep Hough transform for semantic line detection. *IEEE TPAMI*, 2021.

[11] Y. Lin, S. L. Pintea, and J. C. van Gemert. Deep Hough-transform line priors. In *ECCV*, 2020.

[12] J.-Q. Zhang, H.-B. Duan, J.-L. Chen, A. Shamir, and M. Wang. HoughLaneNet: lane detection with deep Hough transform and dynamic convolution. *Comput. Graph.*, 2023.

[13] C. R. Qi, O. Litany, K. He, and L. J. Guibas. Deep Hough voting for 3D object detection in point clouds. In *ICCV*, 2019.

[14] L. Tabelini et al. PolyLaneNet: lane estimation via deep polynomial regression. In *ICPR*, 2020.

[15] R. Liu, Z. Yuan, T. Liu, and Z. Xiong. End-to-end lane shape prediction with transformers. In *WACV*, 2021.

[16] Z. Feng, S. Guo, X. Tan, K. Xu, M. Wang, and L. Ma. Rethinking efficient lane detection via curve modeling. In *CVPR*, 2022.

[17] H. Chen, M. Wang, and Y. Liu. BSNet: lane detection via draw B-spline curves nearby. *arXiv:2301.06910*, 2023.

[18] M. A. Fischler and R. C. Bolles. Random sample consensus. *Commun. ACM*, 24(6):381–395, 1981.

[19] A. Fitzgibbon, M. Pilu, and R. B. Fisher. Direct least square fitting of ellipses. *IEEE TPAMI*, 21(5):476–480, 1999.

[20] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

[21] N. Carion et al. End-to-end object detection with transformers. In *ECCV*, 2020.

[22] TuSimple. TuSimple lane detection benchmark. `https://github.com/TuSimple/tusimple-benchmark`, 2017.